7. CORS – Cross-Origin Resource Sharing

Real-World Example of CORS

- Imagine this scenario:
  - You are inside a company office (this is your browser).
  - You want to access a file from another company (another website/server).
- Now think of a **security guard at that company's gate**:
  - When you request access, the guard checks:
    - "Which company are you from?"
- If your company is allowed:
  - Guard says:
    - "Yes, you can access this resource"
  - You get the file
- If your company is NOT allowed:
  - Guard says:
    - "No access"
  - You are blocked

Mapping to CORS

- Your browser → You
- Other server → Other company
- Origin header → Your company identity
- Access-Control-Allow-Origin → Guard's permission

Same-Origin Policy

- To understand CORS, you must first understand the Same-Origin Policy, which is a built-in browser security rule.
  - It states:
    - JavaScript running on one website **cannot make requests to another website**
  - Definition of an origin:
    - Combination of:
      - Protocol (http/https)
      - Domain (example.com)
      - Port (80, 443, etc.)
  - Example:
    - `https://app.example.com`
    - `https://api.example.com`
    - These are considered **different origins**

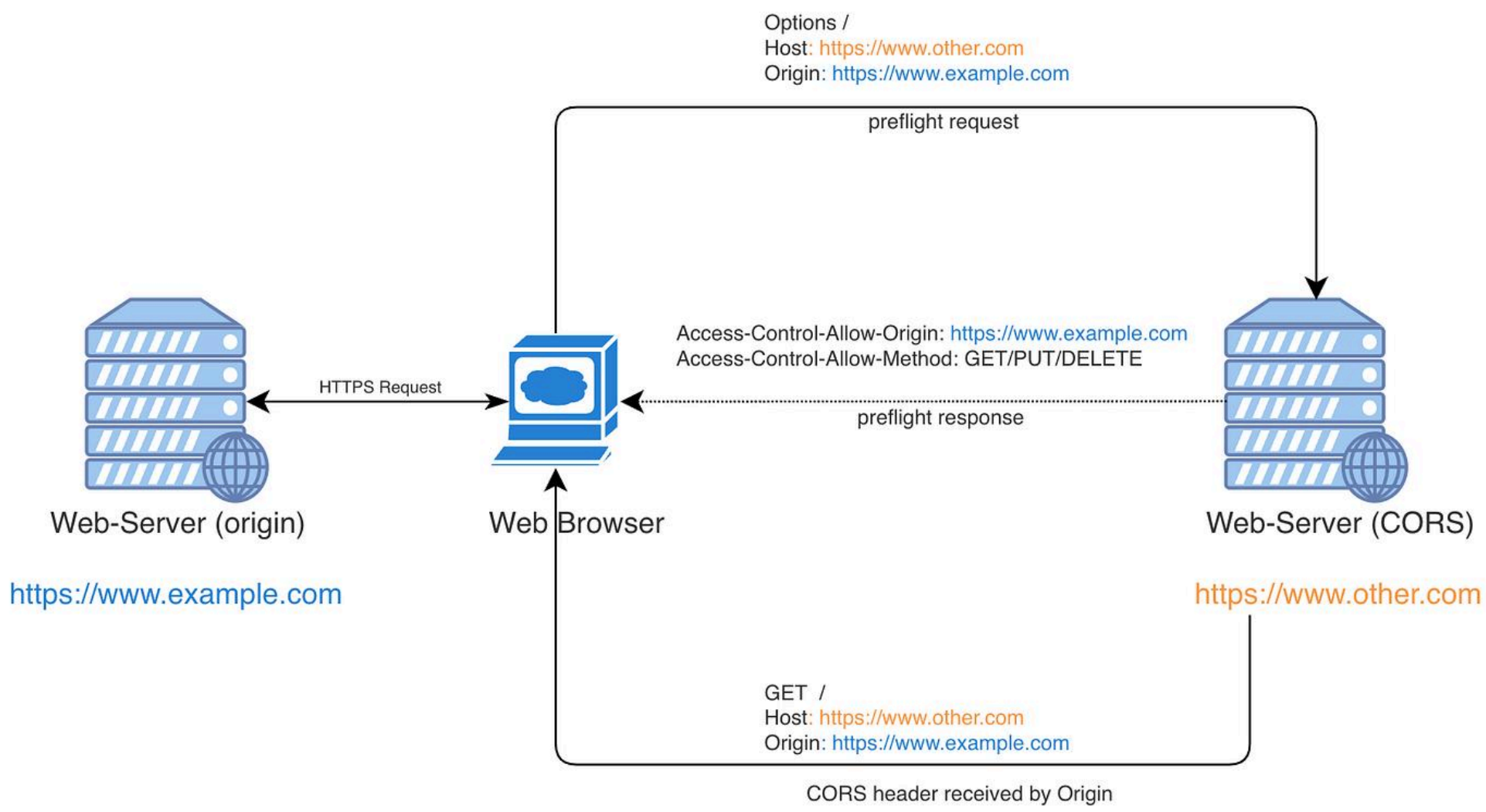
Why Same-Origin Policy Exists

- The Same-Origin Policy is designed for security purposes.
  - It prevents:
    - Malicious websites from making requests to other websites
    - Unauthorized actions using a user's credentials (like cookies or sessions)

Why CORS is Needed

- In real-world applications, frontend and backend are often hosted on different domains.
  - Example:
    - Frontend → `app.example.com`
    - Backend API → `api.example.com`
- Because of Same-Origin Policy:
  - Direct communication would be blocked
- CORS solves this problem by:
  - Allowing servers to explicitly define:
    - Which origins are allowed to access their resources

How CORS Works



- For simple HTTP requests (like GET or POST without custom headers):

Step 1: Browser Sends Request

- Browser automatically adds an **Origin header**.
  - Example:
    - `Origin: https://app.example.com`

Step 2: Server Validates Origin

- Server checks:
  - Whether this origin is allowed

Step 3: Server Responds

- If allowed:
  - Server includes:
    - `Access-Control-Allow-Origin: https://app.example.com`

Step 4: Browser Decision

- If header is present and valid:
  - Browser allows JavaScript to access the response
- If header is missing:
  - Browser blocks the response
  - Even if server returned:
    - `200 OK`

How CORS Works – Complex Requests (Preflight)

- For complex requests such as:
  - Methods:
    - PUT
    - DELETE
  - Or custom headers:
    - Authorization
    - Content-Type (non-standard types)

Step 1: Preflight Request

- Browser sends an **OPTIONS request** before the actual request.
  - Purpose:

        - Ask server:
            - “Am I allowed to send this request?”

Step 2: Server Responds with Permissions

- Server returns CORS headers specifying allowed rules:
    - **Access-Control-Allow-Origin**
        - Specifies allowed origin(s)
    - **Access-Control-Allow-Methods**
        - Specifies allowed HTTP methods
        - Example:
            - GET, POST, PUT, DELETE
    - **Access-Control-Allow-Headers**
        - Specifies allowed headers
        - Example:
            - Authorization, Content-Type
    - **Access-Control-Max-Age**
        - Specifies how long the preflight response can be cached

Step 3: Browser Decision

- If server allows:
    - Browser proceeds with actual request
- If not allowed:
    - Browser blocks the request
    - Shows CORS error

Flow Summary

- Simple request:
    - Direct request → server checks → browser allows/blocks
- Complex request:
    - Preflight (OPTIONS) → server approves → actual request sent

Important Note (Critical for Understanding)

- CORS is enforced by the browser, not the server.
    - Server:
        - Receives and processes the request normally
    - Browser:
        - Decides whether JavaScript can access the response

Key Insight

- CORS is a controlled security mechanism where:
    - The server declares:
        - “These origins are allowed”
    - The browser enforces:
        - Access based on those rules